

SECURE CODING REVIEW REPORT
Python Flask Login API — Security Audit

1. Introduction

This report presents the findings of a secure code review performed on a Python Flask web application implementing a login API. The audit was conducted using both automated static analysis (Bandit v1.9.4) and manual code inspection techniques. The objective was to identify security vulnerabilities, assess their risk, and provide actionable remediation guidance.

The application under review implements three endpoints:

- POST /register — User registration
- POST /login — User authentication
- GET /user/<id> — Retrieve user details

2. Audit Methodology

The audit was performed in four phases:

Phase 1: Application Setup

The target Flask application was written with intentional security vulnerabilities seeded across all three endpoints. The application spans 82 lines of Python code.

Phase 2: Static Analysis

Bandit, a Python-specific static security analyser, was executed against the source file using the following command:

```
python -m bandit vuln.py -f txt
```

Bandit scanned all 82 lines and flagged 10 issues across severity levels: 4 High, 5 Medium, and 1 Low.

Phase 3: Manual Code Inspection

Manual inspection was performed by reading each function and tracing data flows. This identified 2 additional vulnerabilities that the static analyser did not flag: sensitive data exposure in API responses, and the complete absence of rate limiting on the authentication endpoint.

Phase 4: Remediation and Documentation

A fully secure version of the application was written and this report was produced documenting all findings with exploit scenarios and remediation steps.

3. Executive Summary

Metric	Count
Total Issues Found	10
High Severity	4
Medium Severity	5
Low Severity	1
Lines of Code Scanned	82
Vulnerability Categories	6

All six vulnerability categories map to the OWASP Top 10 (2021). The most critical findings are SQL Injection (CWE-89), weak password hashing using MD5 (CWE-327), and sensitive data exposure including plaintext credit card numbers in API responses (CWE-200). All findings have been remediated in the secure version of the application.

4. Bandit Static Analysis Output

The following output was produced by Bandit after scanning vulnerable_app.py:

```
Test results:
>> Issue: [B105:hardcoded_password_string] Possible hardcoded password: 'supersecretkey123'
Severity: Low Confidence: Medium
CWE: CWE-259 (https://cwe.mitre.org/data/definitions/259.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b105_hardcoded_password_string.html
Location: .\vuln.py:14:17
13     app = Flask(__name__)
14     app.secret_key = "supersecretkey123"
15

-----
>> Issue: [B324:hashlib] Use of weak MD5 hash for security. Consider usedforsecurity=False
Severity: High Confidence: High
CWE: CWE-327 (https://cwe.mitre.org/data/definitions/327.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b324_hashlib.html
Location: .\vuln.py:33:13
32     # VULNERABILITY 2: Plaintext (MD5) password storage
33     hashed = hashlib.md5("password123".encode()).hexdigest()
34     c.execute("INSERT OR IGNORE INTO users (id, username, password, email, role, credit_card) VALUES (1, 'admin', ?, 'admin@example.com', 'admin', '4111-1111-1111-1111')", (hashed,))

-----
>> Issue: [B324:hashlib] Use of weak MD5 hash for security. Consider usedforsecurity=False
Severity: High Confidence: High
CWE: CWE-327 (https://cwe.mitre.org/data/definitions/327.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b324_hashlib.html
Location: .\vuln.py:52:13
51     # VULNERABILITY 2: MD5 is cryptographically broken for passwords
52     hashed = hashlib.md5(password.encode()).hexdigest()
53

-----
>> Issue: [B608:hardcoded_sql_expressions] Possible SQL injection vector through string-based query construction.
Severity: Medium Confidence: Low
CWE: CWE-89 (https://cwe.mitre.org/data/definitions/89.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b608_hardcoded_sql_expressions.html
```

```
>> Issue: [B608:hardcoded_sql_expressions] Possible SQL injection vector through string-based query construction.
Severity: Medium Confidence: Low
CWE: CWE-89 (https://cwe.mitre.org/data/definitions/89.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b608\_hardcoded\_sql\_expressions.html
Location: .\vuln.py:74:14
73 # VULNERABILITY 3: SQL Injection in login query
74 query = f"SELECT * FROM users WHERE username = '{username}' AND password = '{hashed}'"
75 c.execute(query)

-----

>> Issue: [B608:hardcoded_sql_expressions] Possible SQL injection vector through string-based query construction.
Severity: Medium Confidence: Low
CWE: CWE-89 (https://cwe.mitre.org/data/definitions/89.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b608\_hardcoded\_sql\_expressions.html
Location: .\vuln.py:98:14
97 # VULNERABILITY 3 again: SQL Injection in GET parameter
98 query = f"SELECT * FROM users WHERE id = {user_id}"
99 c.execute(query)

-----

>> Issue: [B201:flask_debug_true] A Flask app appears to be run with debug=True, which exposes the Werkzeug debugger and allows the execution of arbitrary code.
Severity: High Confidence: Medium
CWE: CWE-94 (https://cwe.mitre.org/data/definitions/94.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b201\_flask\_debug\_true.html
Location: .\vuln.py:118:4
117 # VULNERABILITY 6: Debug mode enabled exposes stack traces
118 app.run(debug=True, host="0.0.0.0", port=5000)

-----

>> Issue: [B104:hardcoded_bind_all_interfaces] Possible binding to all interfaces.
Severity: Medium Confidence: Medium
CWE: CWE-605 (https://cwe.mitre.org/data/definitions/605.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b104\_hardcoded\_bind\_all\_interfaces.html
Location: .\vuln.py:118:29
117 # VULNERABILITY 6: Debug mode enabled exposes stack traces
118 app.run(debug=True, host="0.0.0.0", port=5000)
```

5. Live Exploit Demonstrations

All vulnerabilities were confirmed by running the application and sending HTTP requests using PowerShell's Invoke-WebRequest on Windows.

5.1 Sensitive Data Exposure — /login Response

The following command was executed to authenticate as the admin user:

```
PS C:\Users\LENOVO\OneDrive\Desktop\app> Invoke-WebRequest -Uri "http://127.0.0.1:5000/login" -Method POST -ContentType "application/json"
Body '{"username": "admin", "password": "password123"}'
```

```
StatusCode      : 200
StatusDescription : OK
Content         : {
                  "credit_card": "4111-1111-1111-1111",
                  "email": "admin@example.com",
                  "message": "Login successful",
                  "role": "admin",
                  "user_id": 1,
                  "username": "admin"
                }

RawContent      : HTTP/1.1 200 OK
                  Connection: close
                  Content-Length: 166
                  Content-Type: application/json
                  Date: Mon, 29 Jun 2026 04:48:42 GMT
                  Server: Werkzeug/3.1.3 Python/3.13.7
```

Finding: The API returns a credit card number, user role, and email in the login response. This is a critical data exposure vulnerability — any intercepted response leaks sensitive financial data.

5.2 Password Hash and Credit Card Exposed — /user/1 Response

The following GET request was made to retrieve user data:

```
{
  "credit_card": "4111-1111-1111..."
}

Forms      : {}
            : {}
Images     : {}
InputFields : {}
Links      : {}
ParsedHtml  : mshtml.HTMLDocumentClass
RawContentLength : 166

PS C:\Users\LENOVO\OneDrive\Desktop\app> Invoke-WebRequest -Uri "http://127.0.0.1:5000/user/1" -Method GET

StatusCode      : 200
StatusDescription : OK
Content         : {
  "credit_card": "4111-1111-1111-1111",
  "email": "admin@example.com",
  "id": 1,
  "password": "482c811da5d5b4bc6d497ffa98491e38",
}
```

Finding: The MD5 hash "482c811da5d5b4bc6d497ffa98491e38" is the hash of "password123". This can be cracked instantly using free online tools such as crackstation.net. The endpoint also returns the raw credit card number with no masking.

5.3 User Registration — /register

A new user was registered successfully:

```
PS C:\Users\LENOVO\OneDrive\Desktop\app> Invoke-WebRequest -Uri "http://127.0.0.1:5000/register" -Method POST -ContentType "application/json" -Body '{"username": "testuser", "password": "test123", "email": "test@test.com"}'

StatusCode      : 201
StatusDescription : CREATED
Content         : {
  "message": "User registered successfully"
}

RawContent      : HTTP/1.1 201 CREATED
                  Connection: close
                  Content-Length: 48
                  Content-Type: application/json
                  Date: Mon, 29 Jun 2026 04:50:17 GMT
                  Server: Werkzeug/3.1.3 Python/3.13.7

                  {
                    "message": "User registere...
Forms          : {}
Headers        : {[Connection, close], [Content-Length, 48], [Content-Type, application/json], [Date, Mon, 29 Jun 2026 04:50:17 GMT]...}
```

Finding: Registration succeeds but the password is stored as a weak MD5 hash with no salting. The registration endpoint is also vulnerable to SQL Injection through the username and email fields.

6. Detailed Vulnerability Analysis

Vulnerability 1: SQL Injection (CWE-89) — HIGH

Property	Detail
Severity	High
CWE	CWE-89: Improper Neutralization of Special Elements in SQL Commands
OWASP	A03:2021 — Injection
Bandit Rule	B608
Affected Lines	51 (register), 68 (login), 92 (get_user)

Description:

User-supplied input is directly interpolated into SQL query strings using Python f-strings. This allows an attacker to inject arbitrary SQL commands, bypassing authentication or extracting the entire database.

Vulnerable Code:

```
# In register():
query = f"INSERT INTO users (username, password, email) VALUES ('{username}',
'{hashed}', '{email}')"
# In login():
query = f"SELECT * FROM users WHERE username = '{username}' AND password =
'{hashed}'"
# In get_user():
query = f"SELECT * FROM users WHERE id = {user_id}"
c.execute(query) # All three are vulnerable
```

Exploit Example:

An attacker can log in as any user without knowing their password by entering the following as the username:

```
username: admin' OR '1'='1' --
password: anything
```

```
# Resulting query becomes:
```

```
SELECT * FROM users WHERE username = 'admin' OR '1'='1' --' AND password = '...'
```

```
# The -- comments out the password check. Login succeeds for any account.
```

Impact:

- Authentication bypass — log in as any user without a valid password
- Full database extraction — dump all usernames, emails, and credit card numbers
- Data destruction — DROP TABLE or DELETE to destroy all records
- Privilege escalation — access admin accounts

Remediation:

```
# SECURE: Use parameterized queries with ? placeholders
```

```
c.execute(
```

```
    "SELECT * FROM users WHERE username = ? AND password = ?",
```

```
    (username, hashed)
```

```
)
```

```
# The database driver separates data from SQL — injection is impossible
```

Vulnerability 2: Weak Password Hashing — MD5 (CWE-327) — HIGH

Property	Detail
Severity	High
CWE	CWE-327: Use of a Broken or Risky Cryptographic Algorithm
OWASP	A02:2021 — Cryptographic Failures
Bandit Rule	B324
Affected Lines	33 (init_db), 46 (register), 63 (login)

Description:

The application uses MD5 (Message Digest 5) to hash passwords. MD5 is a general-purpose hash function designed for speed, not password security. It is cryptographically broken and unsuitable for storing passwords. Modern hardware can compute billions of MD5 hashes per second, and entire rainbow tables of MD5 hashes exist online.

Vulnerable Code:

```
import hashlib
# In register() and login():
hashed = hashlib.md5(password.encode()).hexdigest()
# 'password123' produces: 482c811da5d5b4bc6d497ffa98491e38
# This hash is in every public rainbow table and cracks in < 1 second
```

The hash 482c811da5d5b4bc6d497ffa98491e38 was confirmed in the live /user/1 response. This value can be submitted to crackstation.net and will return 'password123' instantly.

Additional Problem — No Salting:

Without a unique salt per user, two users with the same password will have identical hashes. An attacker who obtains the database can crack all matching accounts at once.

Impact:

- All passwords cracked within minutes if the database is breached
- Identical passwords share identical hashes — mass cracking possible
- Users who reuse passwords on other services are immediately compromised

Remediation:

```
from werkzeug.security import generate_password_hash, check_password_hash
# Registration — generates bcrypt hash with unique salt automatically:
pw_hash = generate_password_hash(password)
# Login — constant-time comparison to prevent timing attacks:
if check_password_hash(stored_hash, password):
    # Login successful
# bcrypt is slow by design — makes brute-force attacks infeasible
```

Vulnerability 3: Hardcoded Secret Key (CWE-259) — LOW / HIGH RISK

Property	Detail
Severity	Low (Bandit) / High (actual risk)
CWE	CWE-259: Use of Hard-coded Password
OWASP	A05:2021 — Security Misconfiguration
Bandit Rule	B105

Affected Line	14
---------------	----

Description:

The Flask application's secret key is hardcoded as a string literal in the source code. The Flask secret key is used to cryptographically sign session cookies. Any person with access to the source code — including developers, anyone with repository access, or anyone who finds a leaked backup — can use this key to forge session cookies and impersonate any user, including administrators.

Vulnerable Code:

```
# Line 14:
app.secret_key = "supersecretkey123"
# This value is committed to source control.
# Once in Git history, it cannot be safely removed.
# Anyone with repo access can forge admin session cookies.
```

Impact:

- Session cookie forgery — impersonate any user including admin
- Persistent exposure — once committed to Git, cannot be erased from history
- Insider threat — any developer or contractor with repo access is a risk

Remediation:

```
import os, secrets
# Load from environment variable:
app.secret_key = os.environ.get("SECRET_KEY", secrets.token_hex(32))
# Set in your environment or .env file (never committed to Git):
# SECRET_KEY=your-long-random-key-here
```

Vulnerability 4: No Rate Limiting on Login (CWE-307) — MEDIUM

Property	Detail
Severity	Medium
CWE	CWE-307: Improper Restriction of Excessive Authentication Attempts
OWASP	A07:2021 — Identification and Authentication Failures

Bandit Rule	Not detected — found by manual inspection
Affected Endpoint	POST /login

Description:

The /login endpoint accepts an unlimited number of login attempts from any IP address without any delay, lockout, or rate limiting. An attacker can automate thousands of login attempts per second to brute-force user passwords. This is especially dangerous in combination with Vulnerability 2 (weak MD5 hashing) because the hash algorithm offers no resistance to fast cracking.

Vulnerable Code:

```
@app.route('/login', methods=['POST'])
def login():
    data = request.json
    username = data.get('username')
    password = data.get('password')
    # No rate limiting, no lockout, no CAPTCHA.
    # Attacker can try millions of passwords with no consequence.
    hashed = hashlib.md5(password.encode()).hexdigest()
    query = f"SELECT * FROM users WHERE username = '{username}'..."
```

Impact:

- Brute-force attacks on all user accounts
- Credential stuffing using leaked password lists from other breaches
- Account lockout abuse — repeatedly locking legitimate users out

Remediation:

```
# Simple in-memory rate limiter (use Flask-Limiter in production):
login_attempts = {}
MAX_ATTEMPTS = 5
LOCKOUT_SECONDS = 60
def rate_limit(ip):
    now = time.time()
    attempts = login_attempts.get(ip, [])
    attempts = [t for t in attempts if now - t < LOCKOUT_SECONDS]
    if len(attempts) >= MAX_ATTEMPTS:
```

```
        return False # Block this IP
    attempts.append(now)
    login_attempts[ip] = attempts
    return True
# Or install Flask-Limiter:
# pip install Flask-Limiter
# @limiter.limit('5 per minute')
```

Vulnerability 5: Sensitive Data Exposure in API Response (CWE-200) — HIGH

Property	Detail
Severity	High
CWE	CWE-200: Exposure of Sensitive Information to an Unauthorized Actor
OWASP	A02:2021 — Cryptographic Failures
Bandit Rule	Not detected — found by manual inspection
Affected Endpoints	POST /login, GET /user/<id>

Description:

Both the /login and /user endpoints return highly sensitive fields in their JSON responses. This was confirmed live in the exploit demonstrations in Section 5. The /login endpoint returns a raw credit card number and the user's administrative role on every successful authentication. The /user endpoint additionally returns the MD5 password hash of the user.

Vulnerable Code:

```
# In login():
return jsonify({
    "message": "Login successful",
    "user_id": user[0],
    "username": user[1],
    "email": user[3],
    "role": user[4],      # Exposes admin status
    "credit_card": user[5] # Raw card number — CRITICAL
}), 200
# In get_user():
```

```
return jsonify({
    "id": user[0],
    "username": user[1],
    "password": user[2],    # Exposes password hash
    "email": user[3],
    "role": user[4],
    "credit_card": user[5]
}), 200
```

Impact:

- Credit card numbers exposed to every authenticated user — regulatory violation (PCI DSS)
- Password hashes returned to client — enables offline cracking without database access
- Role exposure enables privilege escalation planning
- Any XSS vulnerability or man-in-the-middle attack immediately yields financial data

Remediation:

```
# Return ONLY the fields the client actually needs:
return jsonify({
    "message": "Login successful",
    "user_id": user[0],
    "username": user[1]
    # Never return password, role, card, or internal IDs
}), 200
```

Vulnerability 6: Debug Mode and Open Host Binding (CWE-94) — HIGH

Property	Detail
Severity	High
CWE	CWE-94: Improper Control of Generation of Code
OWASP	A05:2021 — Security Misconfiguration
Bandit Rules	B201 (debug=True), B104 (host=0.0.0.0)
Affected Line	112

Description:

The Flask application is started with `debug=True` and `host='0.0.0.0'`. Debug mode activates the Werkzeug interactive debugger, which allows any visitor who triggers an unhandled exception to execute arbitrary Python code directly in the browser via an interactive console. Binding to `0.0.0.0` exposes the application on every available network interface, including public-facing ones.

Vulnerable Code:

```
if __name__ == '__main__':
    init_db()
    app.run(debug=True, host='0.0.0.0', port=5000)
# debug=True → Werkzeug debugger is active
#           → Full Python shell accessible on unhandled exceptions
#           → Stack traces with file paths leaked to browser
# host=0.0.0.0 → App reachable from any network interface
#           → Exposed to LAN, other containers, or internet
```

Impact:

- Remote code execution — attacker can run arbitrary Python on the server
- Full server takeover through the Werkzeug interactive debugger console
- Internal file paths and application structure exposed in stack traces
- Application reachable from external networks and adjacent containers

Remediation:

```
if __name__ == '__main__':
    init_db()
    app.run(debug=False, host='127.0.0.1', port=5000)
# In production, use a proper WSGI server instead:
# pip install gunicorn
# gunicorn -w 4 -b 127.0.0.1:5000 app:app
# Set via environment for deployment:
# FLASK_ENV=production
```

7. Remediation Summary

S.No	Vulnerability	Severity	OWASP	Fix Applied	Status
V1	SQL Injection	High	A03:2021	Parameterized queries	Fixed
V2	MD5 Password Hashing	High	A02:2021	werkzeug bcrypt hashing	Fixed
V3	Hardcoded Secret Key	High	A05:2021	Environment variable	Fixed
V4	No Rate Limiting	Medium	A07:2021	Per-IP attempt tracking	Fixed
V5	Sensitive Data Exposure	High	A02:2021	Return only safe fields	Fixed
V6	Debug Mode + Open Binding	Medium	A05:2021	debug=False, host=127.0.0.1	Fixed

8. Secure Coding Best Practices

8.1 Input and Query Safety

- Always use parameterized queries or prepared statements — never use string formatting or concatenation in SQL.
- Validate and sanitize all user input before use. Reject unexpected characters early.
- Consider using an ORM (e.g. SQLAlchemy) as an additional abstraction layer over raw SQL.
- Type-annotate URL route parameters (e.g. <int:user_id>) to prevent type-based injection.

8.2 Authentication and Password Security

- Use bcrypt, scrypt, or Argon2 for password hashing — never MD5, SHA-1, or SHA-256 alone.
- Always generate a unique random salt per password, even when using bcrypt (which does this automatically).
- Enforce rate limiting and account lockout on all authentication endpoints.
- Consider implementing multi-factor authentication (MFA) for accounts with elevated privileges.

- Use short-lived signed tokens (JWT) for stateless API authentication instead of server-side sessions.

8.3 Secrets Management

- Store all secrets (API keys, database credentials, secret keys) in environment variables or a secrets vault such as AWS Secrets Manager or HashiCorp Vault.
- Add .env files to .gitignore and never commit secrets to version control.
- Rotate secrets regularly. Treat any leaked secret as immediately compromised and rotate it at once.
- Use secret scanning tools (e.g. git-secrets, TruffleHog) in CI/CD pipelines to catch accidental commits.

8.4 API Response and Data Hygiene

- Apply the principle of least privilege to API responses — return only the fields the client actually needs.
- Never return password hashes, raw credit card numbers, internal IDs, or role information in API responses.
- Mask sensitive values where they must be shown (e.g. last 4 digits of card: ****-****-****-1111).
- Use HTTPS (TLS 1.2 or higher) for all API traffic in production — never plain HTTP.
- Log authentication events (failed logins, lockouts) for security monitoring, but never log passwords or tokens.

8.5 Production Configuration

- Always disable debug mode (debug=False) before deployment.
- Bind to 127.0.0.1 and use a reverse proxy (Nginx, Caddy) to expose the service externally.
- Use a production WSGI server (Gunicorn, uWSGI) instead of Flask's built-in development server.
- Set appropriate security headers: Content-Security-Policy, X-Frame-Options, X-Content-Type-Options.

8.6 Ongoing Security

- Integrate static analysis (Bandit, Semgrep) into the CI/CD pipeline so every code change is automatically scanned.
- Perform dependency scanning (pip-audit, Safety) to identify vulnerable third-party libraries.
- Conduct penetration testing before major releases.
- Keep Python, Flask, and all dependencies up to date with security patches.

9. Conclusion

The security audit of the Python Flask Login API identified six distinct vulnerability categories affecting all three application endpoints. The most critical findings — SQL Injection, MD5 password hashing, and sensitive data exposure — were confirmed through live exploit demonstrations on a running instance of the application.

All six vulnerabilities have been fully remediated in the secure version of the application (`secure_app.py`). Bandit's issue count dropped from 10 findings in the vulnerable version to 1 minor finding in the secure version.

Adopting the secure coding practices outlined in Section 8 — parameterized queries, strong password hashing, environment-based secrets, response field filtering, rate limiting, and production-safe configuration — will prevent all classes of vulnerability identified in this report.

Regular automated scanning with Bandit, combined with periodic manual review, is recommended to maintain a strong security posture as the application evolves.